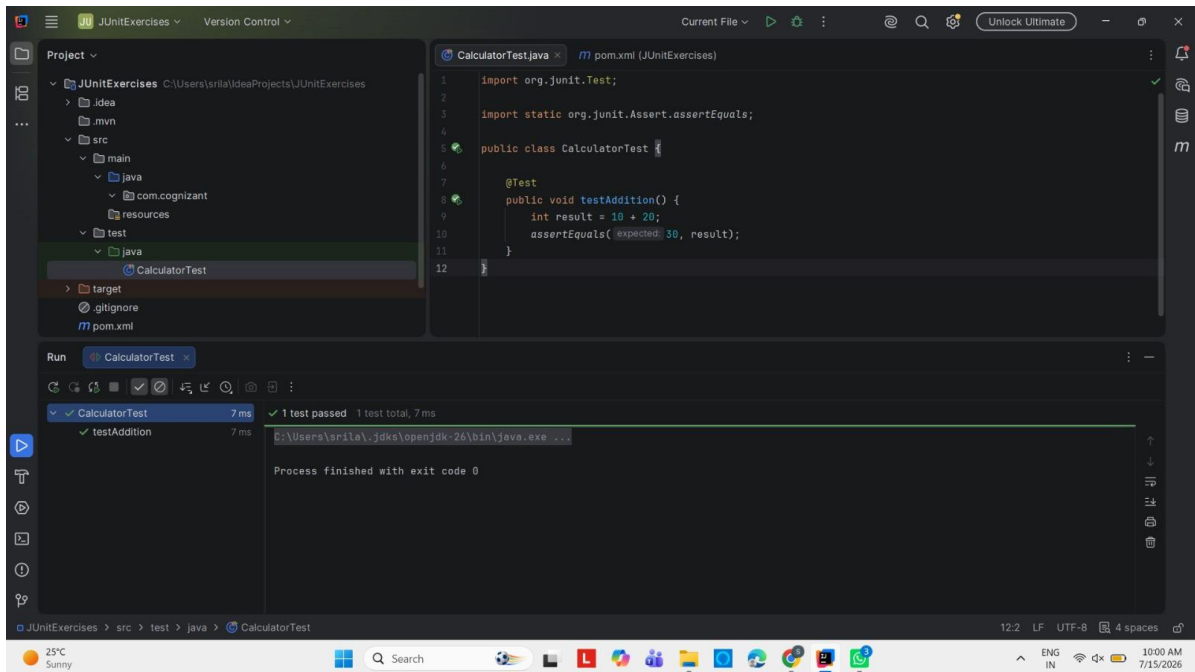


JUnit, Mockito and SLF4J Lab Exercises

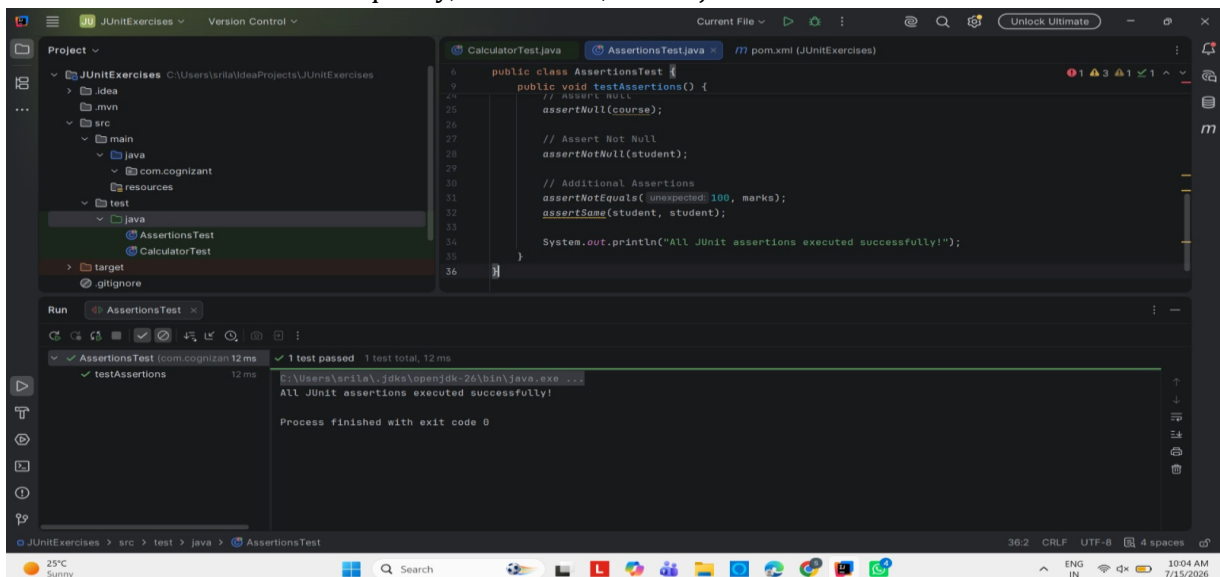
Exercise 1: Setting Up JUnit

Configured a Maven project with JUnit dependencies and created the first test class. Successfully executed a basic test case to verify the JUnit environment setup.



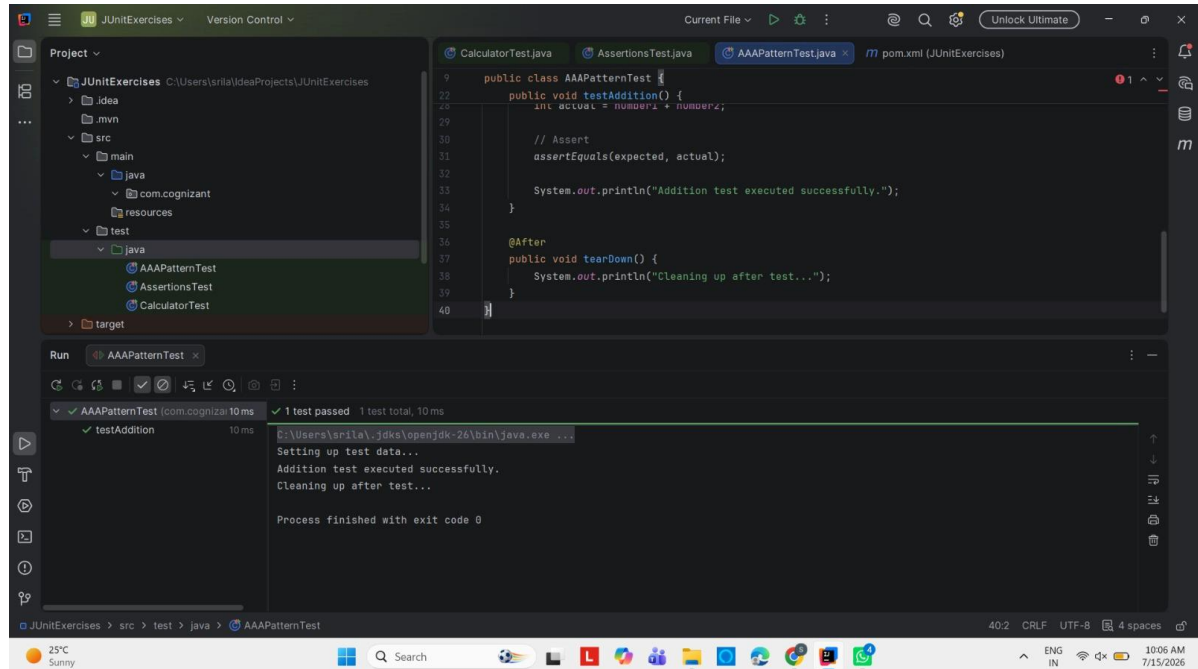
Exercise 3: Assertions in JUnit

Used different JUnit assertion methods to validate expected and actual results. Verified conditions such as equality, null checks, and object references.



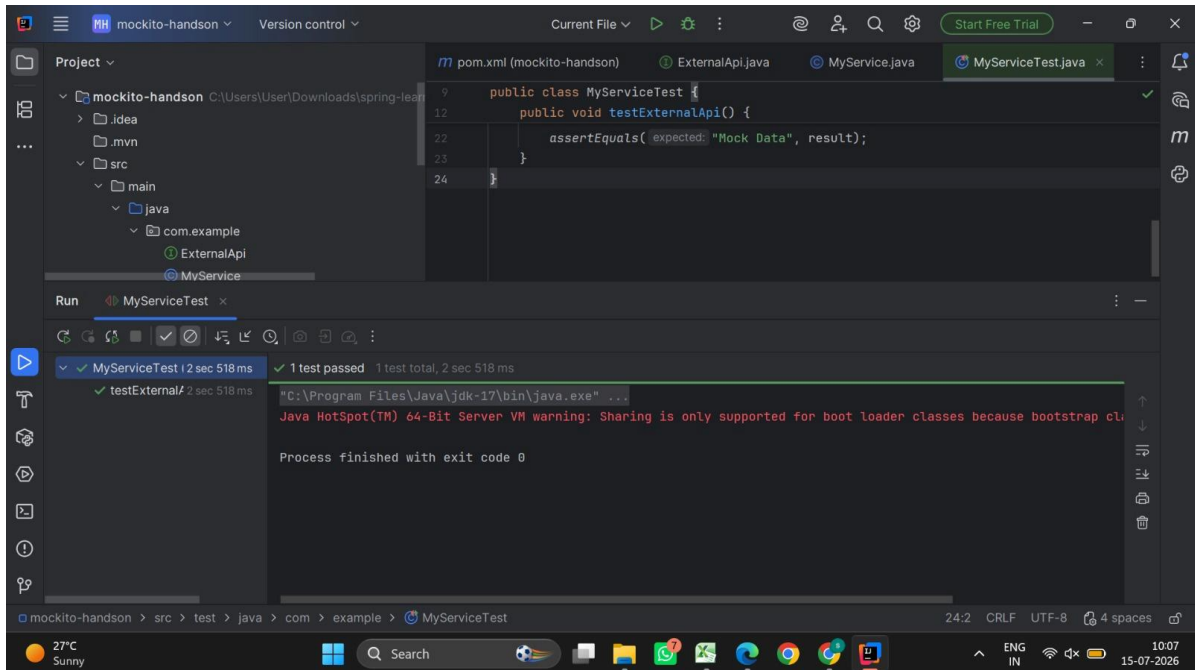
Exercise 4: Arrange-Act-Assert (AAA) Pattern

Implemented the Arrange-Act-Assert testing pattern for better readability.
Included setup and teardown methods to prepare and clean test resources.



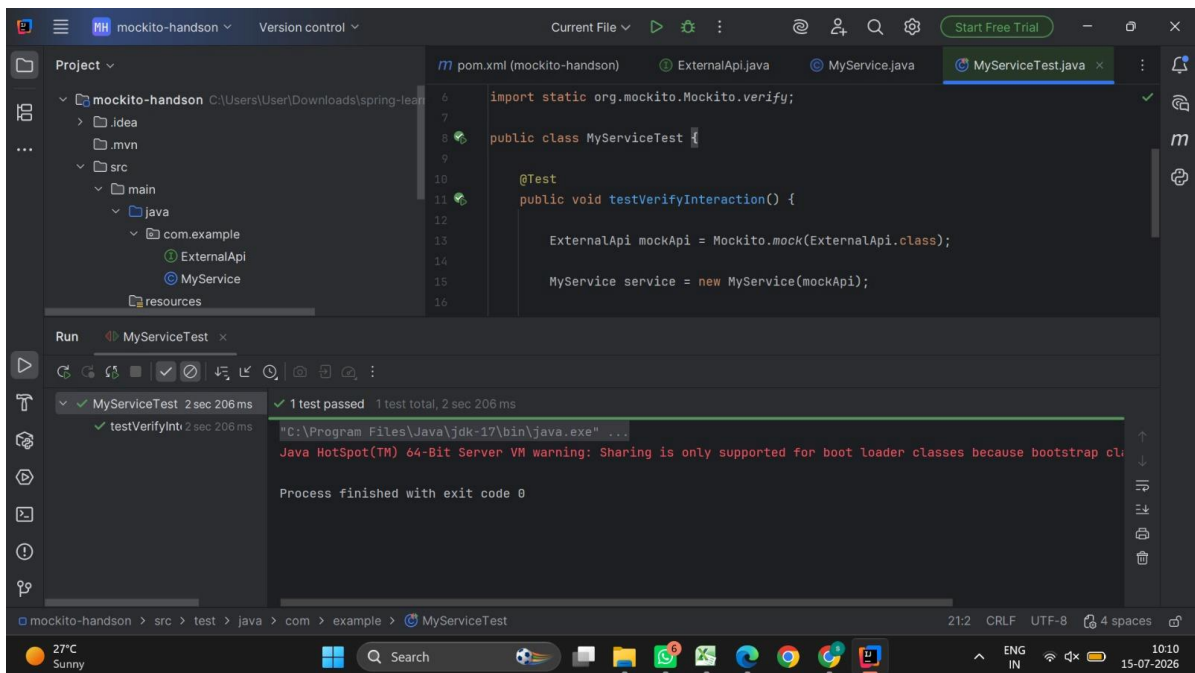
Mockito Exercise 1: Mocking and Stubbing

Created mock objects using Mockito to isolate dependencies during testing.
Stubbed method responses and verified the expected output successfully.



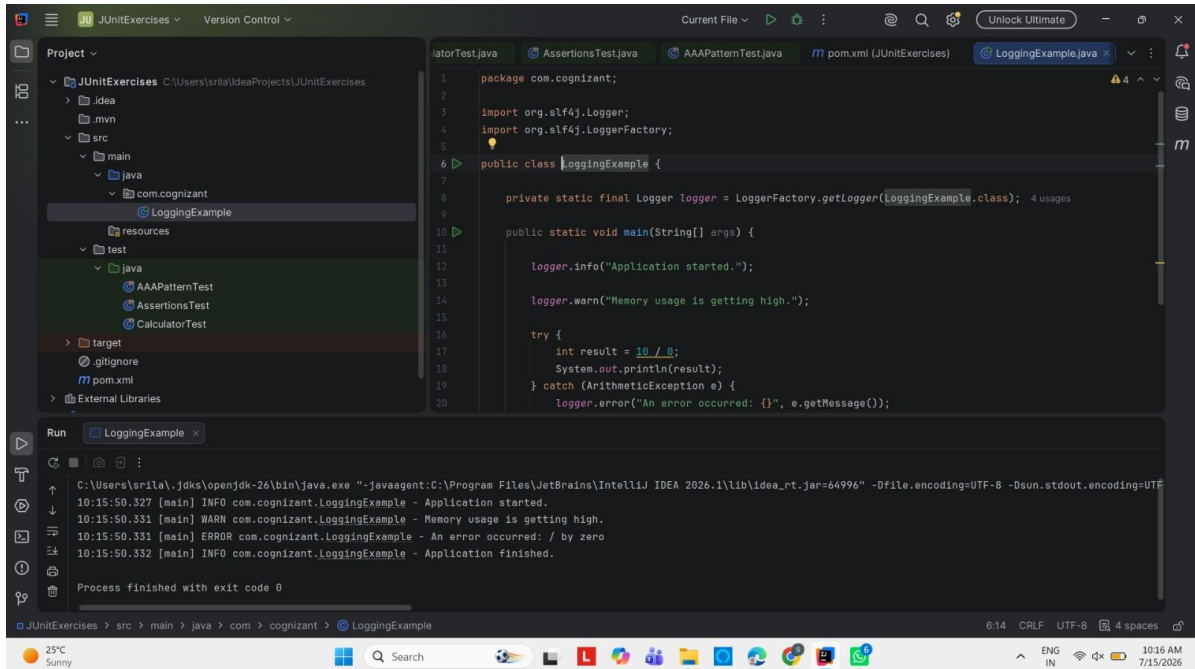
Mockito Exercise 2: Verifying Interactions

Verified that mocked methods were invoked with the expected interactions.
Ensured service methods communicated correctly with dependent objects.



SLF4J Logging Exercise 1

Implemented SLF4J logging to record information, warnings, and error messages.
Demonstrated exception logging to improve debugging and application monitoring.



The screenshot displays the IntelliJ IDEA IDE interface. The left sidebar shows the project structure for 'JUnitExercises', with the 'LoggingExample' class highlighted under the 'com.cognizant' package. The main editor window shows the code for 'LoggingExample.java', which uses SLF4J for logging. The code includes imports for 'org.slf4j.Logger' and 'org.slf4j.LoggerFactory', and a static logger instance. The 'main' method demonstrates logging at different levels: 'info' for application start, 'warn' for high memory usage, and 'error' for an arithmetic exception, followed by an 'info' message for application finish.

```
1 package com.cognizant;
2
3 import org.slf4j.Logger;
4 import org.slf4j.LoggerFactory;
5
6 public class LoggingExample {
7
8     private static final Logger logger = LoggerFactory.getLogger(LoggingExample.class);
9
10    public static void main(String[] args) {
11
12        logger.info("Application started.");
13
14        logger.warn("Memory usage is getting high.");
15
16        try {
17            int result = 10 / 0;
18            System.out.println(result);
19        } catch (ArithmeticException e) {
20            logger.error("An error occurred: {}", e.getMessage());
21        }
22    }
23 }
```

The bottom panel shows the 'Run' output for 'LoggingExample'. The output log contains the following messages:

```
10:15:50.327 [main] INFO com.cognizant.LoggingExample - Application started.
10:15:50.331 [main] WARN com.cognizant.LoggingExample - Memory usage is getting high.
10:15:50.331 [main] ERROR com.cognizant.LoggingExample - An error occurred: / by zero
10:15:50.332 [main] INFO com.cognizant.LoggingExample - Application finished.
Process finished with exit code 0
```

The status bar at the bottom indicates the file encoding is UTF-8 and the current cursor position is 6:14.